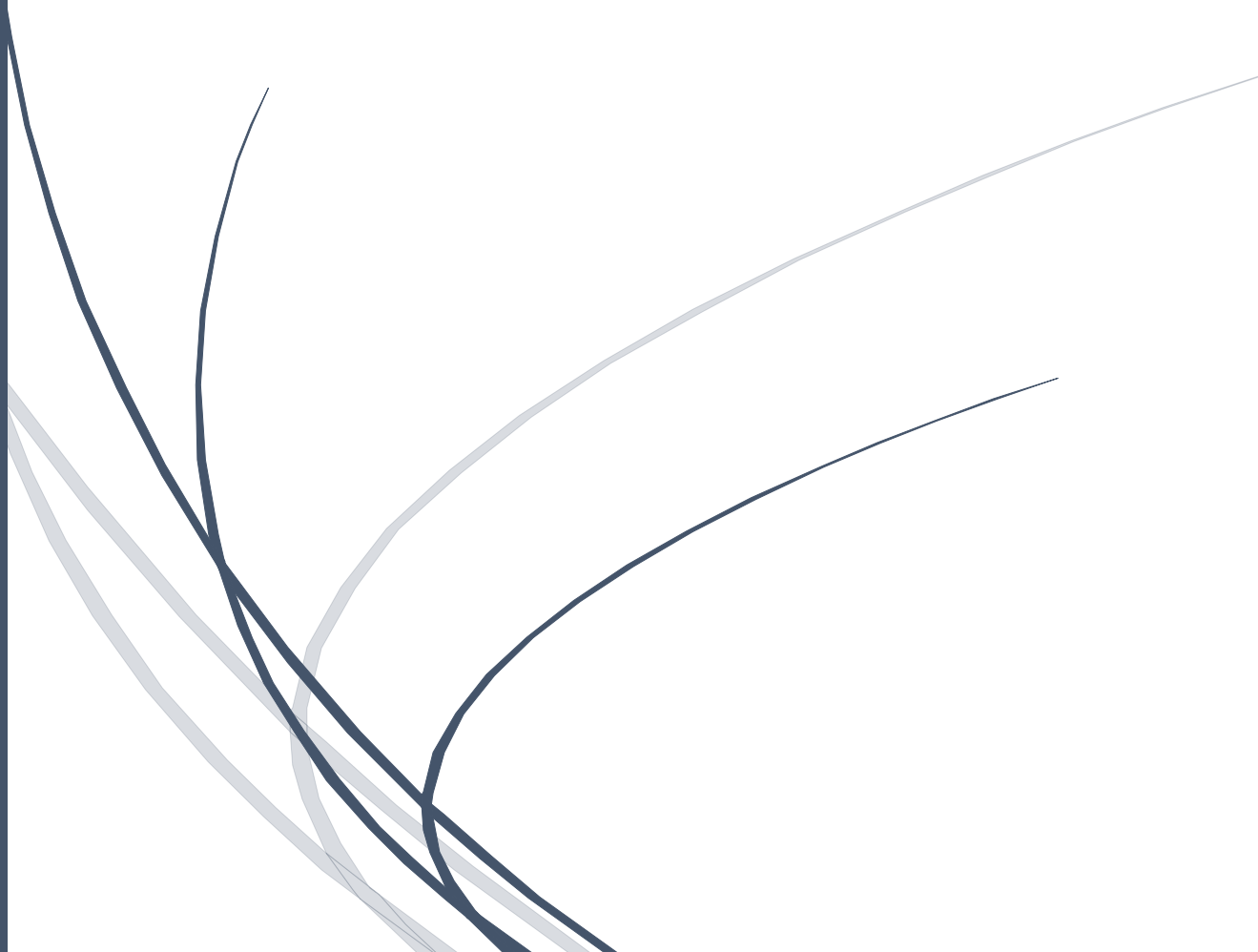


Koszorúalap és fenyőbála készletnyilvántartó webalkalmazás

VAS-DUDÁS DOMINIKA, PROKAI GÁBOR

SZOFTVERFEJLESZTŐ ÉS TESZTELŐ
OKTÁV TOVÁBBKÉPZŐ KÖZPONT
2025-2026



Tartalomjegyzék:

Az alkalmazás célja és követelményei:	2
Az alkalmazás bemutatása:	3
Bejelentkezés:.....	3
Fenyőbála:	4
Koszorúalap:	6
Vásárlók:	7
Az adatbázis.....	8
Felhasználók tábla:	9
Fenyőfajta tábla:	10
Fenyőalap típus tábla:	11
Fenyőbála állapot tábla:	12
Vásárlók tábla:	13
Fenyőbála tábla:	14
Koszorúalap tábla:	15
A Backend	17
A Backend készítésének lépései	18
Előkészületek:	18
Az adatbázis migrálása:	19
Adatok átadása:.....	20
A Backend metódusai	22
A Frontend készítésének lépései	26
Kapcsolat a Frontend és a Backend között	26
A Frontend metódusai.....	27
Az alkalmazás indításához szükséges szoftverek	30

Az alkalmazás célja és követelményei:

Ez az alkalmazás egy jelenleg is működő virágnagykereskedésnek nyújt gyakorlati segítséget a koszorúalapok, és koszorúbálák megrendeléséhez, kezeléséhez. Ebben lehet regisztrálni a vásárlók által megrendelt koszorúalapokat. Az alapokból kétféle, a fenyőből négy fajtát használnak fel a nagykereskedésben. Az alkalmazás abban segít, hogy a vásárlók rendeléseit tudják összegezni, a bálák mennyiségét követni, valamint, hogy melyik fajta fenyő fogy jobban. Mennyit kellett kiszállítani, mennyi lett eladva, lekötve koszorúalapnak, vagy esetleg eltéve egy vásárlónak. Mivel ezek a darabszámok követve vannak - az alkalmazás segítségével - így azt is ki lehet deríteni, hogy mennyi fenyőbála van még raktáron.

A bálákat, a koszorúalapokat, a vevőket tudja kezelni az alkalmazás jelenlegi verziója. Ezen adatok kezeléséhez az alkalmazás mögött egy adatbázisban mindenhez külön táblák készültek. Van olyan tábla, amiben nem változnak az adatok, ezeket „beégetve” kapja meg az adatbázis. Más táblákat viszont a program használatakor tölti fel az alkalmazás használója.

A webalkalmazást kizárólag a nagykereskedés dolgozói használhatják. Saját vevőik egyelőre nem, viszont a lehetőség megvan arra, hogy a vásárlók online leadhassák majd a rendeléseiket. Ez lehet az az irány, amerre az alkalmazás fejleszthető, amennyiben igény van rá. A webes felület könnyen kezelhető még mobiltelefonon is, hiszen a reszponzív felület leköveti a megjelenítő felbontását, és a megfelelő méretre optimalizálja magát.

Jelen állapotban az alkalmazás tudja:

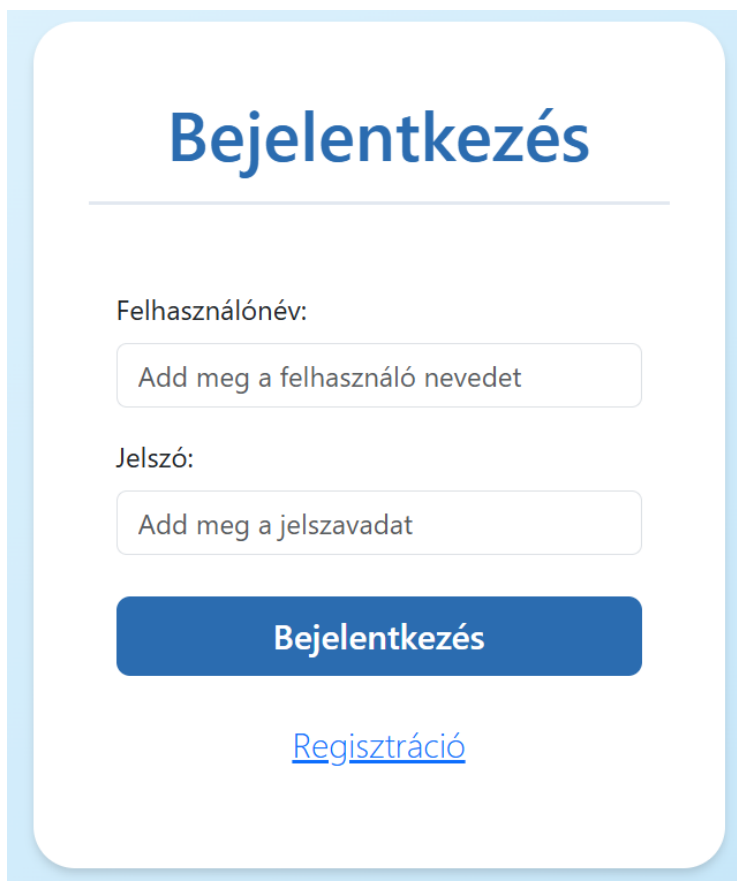
- A felhasználót regisztrálni, beléptetni;
- A koszorúbálák darabszámát követni, jelezni a maradékot;
- Koszorúalapok regisztrációját, valamint, ha elkészült egy alap, azt is tárolja;
- Új vásárlót lehet hozzáadni;

Az alkalmazás bemutatása:

Az alkalmazást – ahogyan már említettem – csak a nagykereskedelem alkalmazottjai használhatják. A kezdőlapra a **Bejelentkezés** oldal jelenik meg a felhasználó előtt. Új felhasználó hozzáadásának lehetősége is beépítésre került az oldalra, arra az esetre, ha új dolgozót vesz fel a nagykereskedelem.

Bejelentkezés:

A felhasználónév és jelszó kombinációk megadásával a felhasználó be tud lépni a következő felületre, ahonnan lehetősége nyílik a vevők, a koszorúalapok, valamint a fenyőbálák adatainak módosítására, regisztrációjára, törlésére stb.

A képen a Bejelentkezés felület látható. A cím "Bejelentkezés" nagy, kék betűkben van megadva. Alatta két mező található: "Felhasználónév:" és "Jelszó:". Mindkét mezőben a "Add meg a felhasználó nevedet" illetve "Add meg a jelszavadat" szöveg látható. A mezők alatt egy kék gomb van, amelyen "Bejelentkezés" van írva. A gomb alatt egy kék link található, amelyen "Regisztráció" van írva.

Bejelentkezés

Felhasználónév:

Add meg a felhasználó nevedet

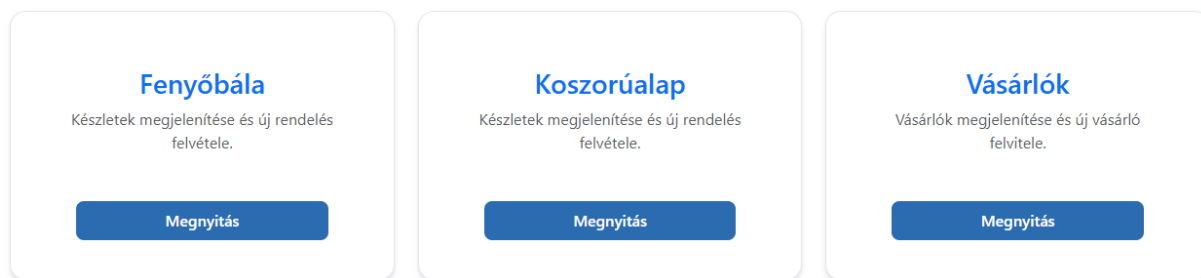
Jelszó:

Add meg a jelszavadat

Bejelentkezés

[Regisztráció](#)

Megfelelő adatok megadása után a **Lehetőségek** oldalra navigál minket az alkalmazás. Ezen az oldalon lehet kiválasztani, hogy éppen mivel akarunk foglalkozni.



Fenyőbálát rendelni, vagy koszorúalapot felírni egy vásárlónak, vagy egy új vevőt regisztrálni.

Fenyőbála:

Ezen az oldalon lehet hozzáadni a rendelések adatait. Új bála hozzáadása esetén ki kell választani a fenyő típusát, az állapotot, valamint a darabszámot. A fenyő típusát-, és állapotát egy-egy legördülő mező lehetőségiből kell kiválasztani. Ezek a lehetőségek szintén az adatbázisból kerülnek elő, mert ezek előre beírt, megadott adatok. A **Mentés** gombra kattintva az alkalmazás – a megadott adatokat – az adatbázisban tárolja.

A form titled 'Új rendelés' with a light blue border. It contains a section 'Új bála hozzáadása:' with three fields: 'Fenyő fajta:' (dropdown menu with '-- Válassz --'), 'Állapot' (dropdown menu with '-- Válassz --'), and 'Darabszám:' (text input field). Below these fields is a blue 'Mentés' button.

Az oldal másik 2/3-os részében jelenik meg a nagykereskedés raktári összesítője. A bála hozzáadása után az adatbázisban tárolt mennyiségek ebben a táblázatban jelennek meg.

Raktári adatok				
Állapot	Lucfenyő	Jegenye	Normand	Nobilis
Érkezett	250	100	120	125
Kiszállítva	140	10	0	10
Eladva	60	5	5	0
Lekötve	58	20	10	0
Eltéve	2	2	2	2
Maradt	-10	63	103	113

Az állapot mezőben a legtöbbet alkalmazott „tárolási” lehetőségek vannak felsorolva. Ezek az állapotok fordulnak elő a legtöbbször az üzletben. A többi mező a fenyők fajtáit adja meg, amit a nagykereskedés felhasznál.

A táblázat többi része már a feltöltött adatbázis adatait mutatja meg a felhasználónak. Amikor egy bála rendelés beérkezik, akkor az a mennyiségű bála az „Érkezett” sorba kerül. Természetesen a megfelelő oszlopba teszi az adatbázis, mert nem csak az állapotot kell megadni, hanem a fenyőbála típusát is.

Amikor az alkalmazást megfelelően használják, akkor az adatok a megfelelő helyükre kerülnek, és akkor a valódi raktárban és a „digitalizált” raktárban a fenyőbálák mennyisége, és az adatok is megegyeznek.

Koszorúalap:

Ez az oldal is 1/3-2/3 arányban fel van osztva. Az 1/3 részen szintén az adatok felvitelére van lehetőség. Az összes beviteli mező kitöltése kötelező, hiszen fontos adatokról van szó. A vevő, a koszorúalap mérete, a fenyő fajtája, a határidő – ami a vevőnek nagyon fontos – és a darabszám, hogy mennyi alapra van szükség. Az összes beviteli mező itt is legördülő elem, és az adatbázisból kisorsolt adatok jelennek meg benne választható lehetőségként. A megadott adatokat a **Mentés** gombra kattintva tárolja az adatbázis.

Új rendelés

Új rendelés hozzáadása:

Vásárló

-- Válassz --

Fenyő fajta

-- Válassz --

Alap típus:

-- Válassz --

Határidő

éééé. hh. nn. 

Darabszám:

Mentés

Az oldal 2/3-ad részében az előbb megadott adatok szerepelnek, csak itt már olyan táblázatos formában, mit a már említett fenyőbála rendelés esetében. A mezőnevek természetesen mások, mert itt más adatokat kellett megadni.

Rendelés részletei					
Kinek	Mikorra	Fenyő fajta	Alap típus	Mennyiség	Kész
Elek	2026-09-07.	Nobilis	Nagy	5	☒
Kiss Ferencz	2026. 09. 09.	Normand	Kicsi	5	☐
Béla	2026. 09. 30.	Luc	Nagy	10	☐
Géza	2026-09-16.	Jegenye	Kicsi	10	☒

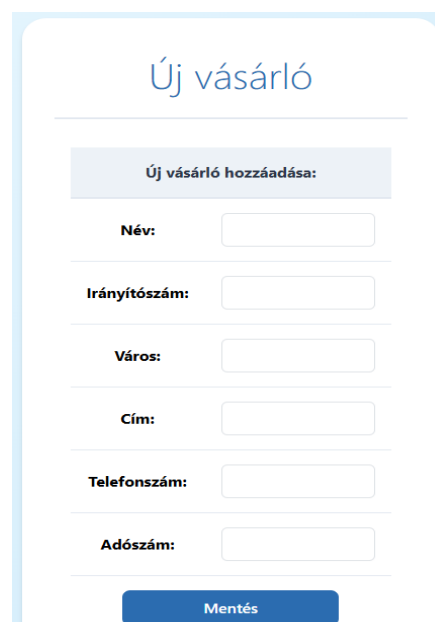
A táblázat fejléce ennél az oldalnál is a bekért adatokat tartalmazza. Az adatok bekérésénél a koszorúalap még nincs kész állapotban, ezért a kész oszlop jelölőnégyzete még

hamis állapotban van. Amint a koszorúalap készen van, a jelölőnégyzetre kattintással lehet jelezni az adatbázisnak, hogy a koszorúalap elkészült.

A **Kinek** a vásárlót jelenti, a **Mikorra** a határidőt, a **Fenyő fajta** az négy fajta lehet, az **Alap típus** két lehetőség közül lehet választani, a **Darabszám** is fontos, a **Kész** pedig az adatbázisnak, és a program felhasználójának ad fontos információt a koszorúalap állapotáról.

Vásárlók:

Ezen az oldalon a vásárlókat lehet regisztrálni az adatbázisba. Az oldal a már ismerős 1/3-2/3-ad felosztásban van. Az „Új vásárló hozzáadása” részben minden olyan információ, ami a vevő azonosításához szükséges fel van sorolva az űrlapon. A megfelelő adatok kitöltésével, majd a „Mentés” gombra kattintással a vevőt az adatbázisba menti a program.



Új vásárló

Új vásárló hozzáadása:

Név:

Irányítószám:

Város:

Cím:

Telefonszám:

Adószám:

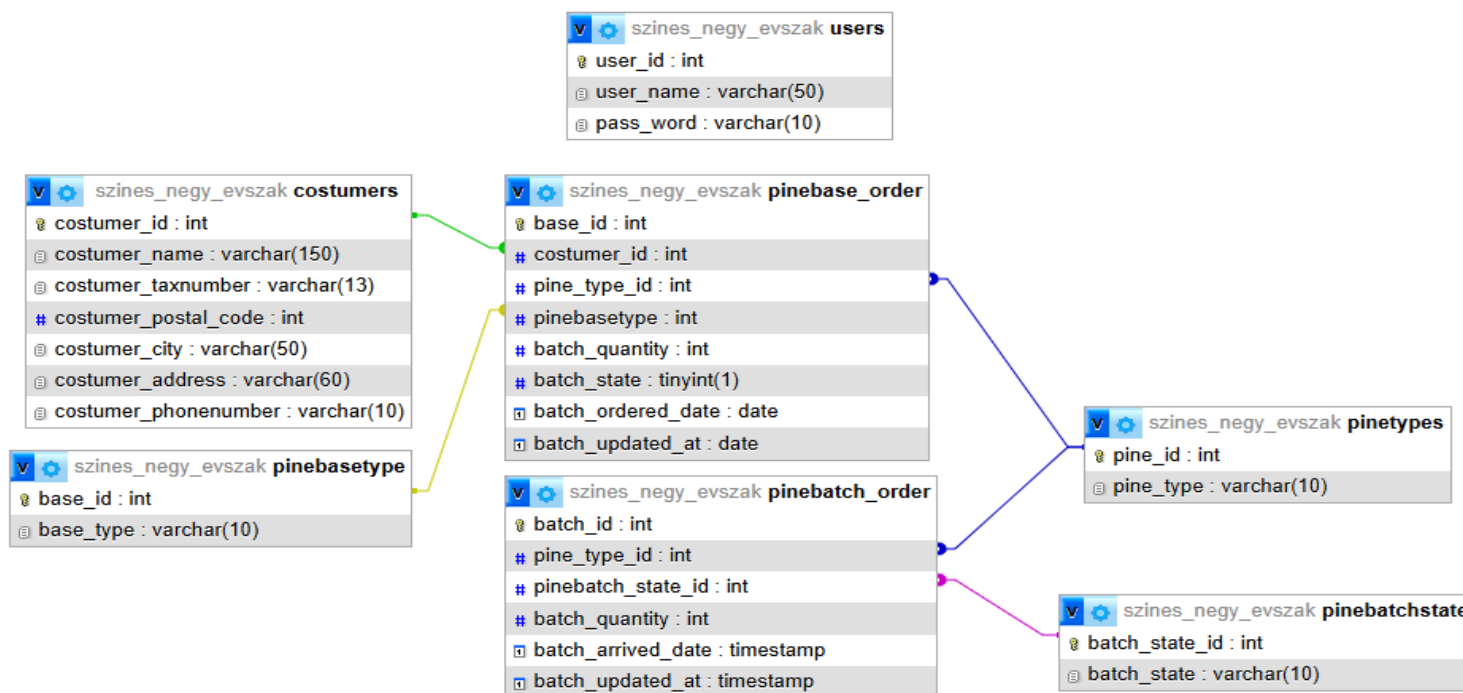
Mentés

A regisztrált vásárlók adatait az oldal másik része jeleníti meg – szintén – táblázatos formában. Az adatok itt csak megjelenítve vannak, de a későbbiek folyamán majd az adatok módosítására is lesz lehetőség.

Regisztrált vásárlók adatai					
Név	Irányítószám	Város	Cím	Telefonszám	Adószám
Adatok betöltése folyamatban...					

Az adatbázis

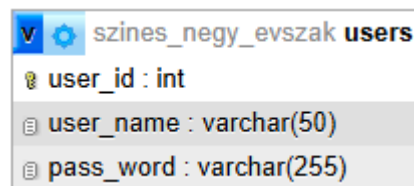
Az előbb bemutatott webalkalmazás adatait egy adatbázisban tároljuk. Jelen esetben ezt a „Wampserver” rendszer segítségével hoztuk létre. Az adatbázis hét táblát tartalmaz, melyekben az egymással összefüggő adatokat tároljuk. Az adatbázis szerkezetét az alábbi ábra mutatja.



Felhasználók tábla:

Ebben, ahogyan a tábla neve is mondja, a felhasználók adatait tároljuk le. Ehhez a következő mezőket kellett létrehozni:

- **user_id** – Egy int típusú változó, aki az elsődleges kulcsa a táblának és automatikusan növekszik az értéke;
- **user_name** – Egy varchar típusú változó. Természetesen a felhasználó nevét tartalmazza;
- **pass_word** – Szintén egy varchar változó. A felhasználó jelszavát tartalmazza, viszont a jelszót „hash” -elve tárolja;



	szines_negy_evszak	users
🔑	user_id	int
📄	user_name	varchar(50)
📄	pass_word	varchar(255)

A tábla tervező nézetben látható a fenti képen, a forráskódja pedig lentebb található:

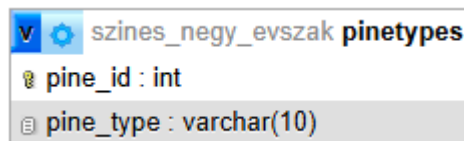
```
DROP TABLE IF EXISTS `users`;  
CREATE TABLE IF NOT EXISTS `users` (  
  `user_id` int NOT NULL AUTO_INCREMENT,  
  `user_name` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,  
  `pass_word` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,  
  PRIMARY KEY (`user_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Fenyőfajta tábla:

A nagykereskedés négy fajta fenyőt használ fel, illetve ad el. Ezek tárolásához is létre lett hozva egy tábla. A táblázat adatai előre meg vannak adva, viszont a későbbi típusok hozzáadása is lehetséges lesz majd később.

A mezők, amiket a tábla tartalmaz:

- **pine_id:** int típusú, elsődleges kulcsa a táblának, mely automatikusan növekszik;
- **pine_type:** varchar típusú mező, mely a fenyőfajta nevét tartalmazza;



A tábla tervező nézetben. Ennek a táblának az adatait még két másik tábla is fogja használni, ahol majd az itteni elsődleges kulcs a másik táblában másodlagos kulcsként szerepel majd. Lentebb a tábla létrehozásának és az adatok beillesztésének forráskódja látható:

```
DROP TABLE IF EXISTS `pinetypes`;
CREATE TABLE IF NOT EXISTS `pinetypes` (
  `pine_id` int NOT NULL AUTO_INCREMENT,
  `pine_type` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  PRIMARY KEY (`pine_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

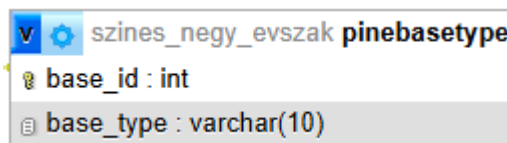
INSERT INTO `pinetypes` (`pine_id`, `pine_type`) VALUES
(1, 'Luc'),
(2, 'Jegenye'),
(3, 'Normand'),
(4, 'Nobilis');
COMMIT;
```

Fenyőalap típus tábla:

Az előző táblánál már említve lett, hogy négyfajta fenyőt használnak fel. Mind a négy típusból érkeznek bálák. Ezekből kötnek koszorúalapot, valamint el is adnak belőle. Viszont ez a tábla csak a koszorúalaphoz kötődik oly módon, hogy a koszorúalapból a nagykereskedés csak kettő félét köt. Ezen adatok vannak a táblában az előzőhöz hasonlóan beégetve. Egyébként a könnyebb adatbevitel érdekében lett ez létrehozva.

A tábla mezői:

- **base_id**: int típusú, a tábla elsődleges kulcsa, ami automatikusan növekszik;
- **base_type**: varchar típusú mező, mely az alap típusát tartalmazza;



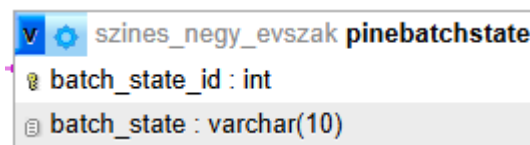
Tervező nézetben a tábla. Ennek a táblának egy másik tábla fogja használni az adatait. Természetesen ott másodlagos kulcsként szerepel az itteni elsődleges kulcs. Lentebb a tábla létrehozásának forráskódja -, és az adatok beégetéséhez szükséges forráskód látható.

```
DROP TABLE IF EXISTS `pinebasetype`;  
CREATE TABLE IF NOT EXISTS `pinebasetype` (  
  `base_id` int NOT NULL AUTO_INCREMENT,  
  `base_type` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  PRIMARY KEY (`base_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
INSERT INTO `pinebasetype` (`base_id`, `base_type`) VALUES  
(1, 'Kicsi'),  
(2, 'Nagy');
```

Fenyőbála állapot tábla:

A nagykereskedés által rendelt fenyőbálákat többféle módon kezelik. A tábla ezen adatokat tartalmazza. Rendelés után **érkezik**, van olyan, amit **lekötnek**, amit **eladnak**, amit **kiszállítanak**, esetleg valakinek **eltesznek**. Ezeket az állapotokat tartalmazza a tábla az alábbiak szerint:

- **batch_state_id**: int típusú elsődleges kulcsa a táblának, automatikusan növekszik;
- **batch_state**: varchar típusú, az állapot szöveges definíciója;



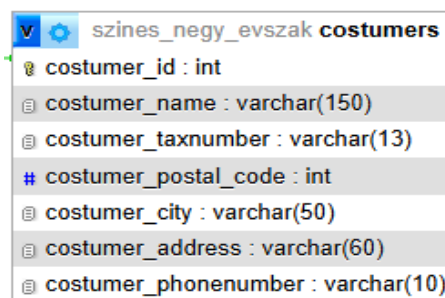
A tábla tervező nézete a fenti ábra. Lentebb található a tábla létrehozásának kódja, valamint a táblázatot feltöltő forráskód is.

```
DROP TABLE IF EXISTS `pinebatchstate`;  
CREATE TABLE IF NOT EXISTS `pinebatchstate` (  
  `batch_state_id` int NOT NULL AUTO_INCREMENT,  
  `batch_state` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  PRIMARY KEY (`batch_state_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
INSERT INTO `pinebatchstate` (`batch_state_id`, `batch_state`) VALUES  
(1, 'Érkezett'),  
(2, 'Elvitt'),  
(3, 'Eladva'),  
(4, 'Lekötve'),  
(5, 'Eltéve');
```

Vásárlók tábla:

A vásárlóknak is van egy külön tábla, melyben az adataikat tároljuk. Itt már jóval több adatra van szükség, főleg, ha a későbbiek folyamán majd hivatalossá is lehetne tenni az alkalmazást. Jelen esetben a vásárlók nem tudnak regisztrálni az alkalmazásba, viszont a nagykereskedés dolgozói fel tudják venni az adataikat. A táblában megtalálható adatok a következők:

- **costumer_id**: int típusú, elsődleges kulcs, automatikusan növekszik;
- **costumer_name**: varchar típusú, a vevő nevét tartalmazza;
- **costumer_taxnumber**: a vevő adószáma;
- **costumer_postal_code**: az irányítószám, ahol a vevőnek van a boltja;
- **costumer_city**: a város, ahol a vevő üzlete van;
- **costumer_address**: az üzlet címe;
- **costumer_phonenumber**: az üzlet telefonszáma;



szines_negy_evszak costumers
costumer_id : int
costumer_name : varchar(150)
costumer_taxnumber : varchar(13)
costumer_postal_code : int
costumer_city : varchar(50)
costumer_address : varchar(60)
costumer_phonenumber : varchar(10)

A tábla tervező nézetben, alatta a tábla létrehozásához szükséges forráskód látható.

```
/*A vevők adatait tartalmazó tábla létrehozása*/  
CREATE TABLE costumers (  
    costumer_id int PRIMARY KEY AUTO_INCREMENT,  
    costumer_name varchar(150),  
    costumer_taxnumber varchar(13),  
    costumer_postal_code int,  
    costumer_city varchar(50),  
    costumer_address varchar(60),  
    costumer_phonenumber varchar(10)  
);
```

Fenyőbála tábla:

A nagykereskedésnél a fenyőbála nagyon fontos alapanyag. Ebből kötnek koszorúalapot, valamint még eladnak, kiszállítják, és sokféleképpen kezelik az érkezett bálákat. A bálák négyfajta fenyőből hetente érkeznek. Az érkezett bálák számát felvezetik az adatbázisban, majd a hét folyamán – amikor kiszállítás van, vagy lekötnek koszorúalapokat, vagy egy-két vevőnek eltesznek – azokat ez a tábla tartalmazza. Itt már számolni kell a megmaradt bálákat. A tábla a következőképpen néz ki:

- **batch_id**: int típusú, elsődleges kulcs, automatikusan növekszik;
- **Pine_type_id**: int típusú mező. Itt másodlagos kulcsként szerepel. Ez a pinetypes tábla elsődleges kulcsa;
- **pinebatch_state_id**: int típusú mező. Ez is másodlagos kulcs. A pinebatchstate tábla elsődleges kulcsa;
- **pinebatch_quantity**: int típusú mező. Az érkezett fenyőbálák mennyiségét tartalmazza;
- **batch_arrived_date**: date típusú mező. Ez az érték automatikusan íródik be az adatbázis táblájába, amikor a megadott adatok mentésre kerülnek.
- **batch_updated_at**: date típusú mező. Ennek az értéknek akkor kell új dátumot kapnia, amikor a fenyőbála állapota nem az érkezett állapotot veszi fel.

A tábla szerkesztő nézete, és a létrehozásához szükséges kód:

```
szines_negy_evszak pinebatch_order
batch_id : int
# pine_type_id : int
# pinebatch_state_id : int
# batch_quantity : int
batch_arrived_date : date
batch_updated_at : date
```

```

DROP TABLE IF EXISTS `pinebatch_order`;
CREATE TABLE IF NOT EXISTS `pinebatch_order` (
  `batch_id` int NOT NULL AUTO_INCREMENT,
  `pine_type_id` int DEFAULT NULL,
  `pinebatch_state_id` int DEFAULT NULL,
  `batch_quantity` int DEFAULT NULL,
  `batch_arrived_date` date DEFAULT NULL,
  `batch_updated_at` date DEFAULT NULL,
  PRIMARY KEY (`batch_id`),
  KEY `fk_pinebatch_pinetype` (`pine_type_id`),
  KEY `fk_pinebatch_state_id` (`pinebatch_state_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Koszorúalap tábla:

A fenyőbála egyik felhasználási állapotát tárolja ez a tábla. A nagykereskedésben a bálák egyik felhasználási módja a koszorúalap kötés. A koszorúalapból kétfélét készítenek, és a fenyőkből is négy fajta van, amit felhasználnak. Valamint a koszorúalapot egy regisztrált vásárlónak kell megrendelnie. Ezért az adatokat tartalmazó táblát a következőképpen kellett kialakítani:

- **base_id**: int típusú, a tábla elsődleges kulcsa, automatikusan növekszik;
- **costumer_id**: int típusú, a tábla egyik másodlagos kulcsa, a vásárlót azonosítja;
- **pine_type_id**: int típusú, a tábla szintén másodlagos kulcsa, a fenyő fajtájára mutat;
- **pinebasetype**: int típusú, a tábla harmadik másodlagos kulcsa, a koszorúalap méretét mutatja meg;
- **base_quantity**: int típusú mező, a megrendelt alapok mennyiségét tárolja;
- **base_state**: igazából bool típusú, ez ahhoz kell, hogy a koszorúalap elkészült, vagy még nem;
- **base_ordered_date**: date típusú mező, ez tárolja azt a dátumot, amikor a koszorúalapnak készen kell lennie;
- **base_updated_at**: date típusú, ez akkor változik meg, amikor az alap elkészül, és a checkbox állapota megváltozik;

szines_negy_evszak pinebase_order	
base_id	int
# costumer_id	int
# pine_type_id	int
# pinebasetype	int
# base_quantity	int
# base_state	tinyint(1)
base_ordered_date	date
base_updated_at	date

```

DROP TABLE IF EXISTS `pinebase_order`;
CREATE TABLE IF NOT EXISTS `pinebase_order` (
  `base_id` int NOT NULL AUTO_INCREMENT,
  `costumer_id` int DEFAULT NULL,
  `pine_type_id` int DEFAULT NULL,
  `pinebasetype` int DEFAULT NULL,
  `base_quantity` int DEFAULT NULL,
  `base_state` tinyint(1) DEFAULT NULL,
  `base_ordered_date` date DEFAULT NULL,
  `base_updated_at` date DEFAULT NULL,
  PRIMARY KEY (`base_id`),
  KEY `fk_costumer_id` (`costumer_id`),
  KEY `fk_pinebase_pinetype` (`pine_type_id`),
  KEY `fk_pinebasetype` (`pinebasetype`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

A Backend

A Backend megírásához a Microsoft Visual Studio-t használtuk. A backend szoftvert C# nyelven írtuk meg. A Visual Studio alaphelyzetben nem tartalmazza azokat a programcsomagokat, mellyel a Frontend-et sikeresen ki tudja szolgálni, ezért azokat a NuGet Package Manager-rel kellett külön telepíteni. A programcsomagok, amiket a backend használatához, teszteléséhez használunk és telepítettünk fel:

BCryp.Net-Next: Ezen programcsomag segítségével lehet a jelszavakat úgy titkosítani, hogy az adatbázisban nem a jelszó jelenik meg, hanem egy „hash” -elt karakterkód;

Microsoft.AspNetCore.OpenApi: Ez a csomag összegyűjti a http végpontokkal kapcsolatos információkat közvetlenül a kódból;

Microsoft.EntityFrameworkCore.Tools: Ez a Visual Studio Console ablakába integrálja a NuGet Package Manager-t, amit az adatbázis migrálásához használtunk fel a projekt fejlesztése során;

OpenAPI.NET: Ez az egységes objektummodellezéshez kell. Ennek a segítségével lehet C# osztályokat kezelni, amelyek leképezik egy HTTP API leíró fájl struktúráját. Ezek a fájlok tartalmazzák az útvonalakat, a paramétereket és a válaszokat. Itt szigorúan csak a 2.8.0-ás verzió jöhet számításba, a fejlesztés során valamiért a legfrissebb mindig hibaüzenettel tért vissza migrálás helyett;

MySQL.EntityFrameworkCore: Ez a segédprogram pedig hídként szolgál a .NET-alkalmazások és a MySQL adatbázisok között. Az adatbázis tábláit C# osztályokká, valamint a sorokat pedig objektummá tudja konvertálni mindkét irányba természetesen;

Scalar.AspNetCore: Végül pedig a teszteléshez egy nagyon hasznos kiegészítő szoftver, melynek segítségével a http végpontok tesztelhetők még a frontend elkészítése előtt. Viszont akkor is jó célt szolgál, amikor megosztott munkavégzés van. Akkor a fejlesztőknek csak ugyanazokat a végpontokat kell definiálniuk, és a frontend-, valamint a backend párhuzamosan fejleszthető;

A Backend készítésének lépései

Előkészületek:

Miután az előbb felsorolt segédprogramokat feltelepítettük a projektünkhöz hozzákezdhetünk a programunk fejlesztéséhez. Ehhez elsősorban a segédprogramok behívási pontjait adjuk meg a megfelelő forráskódban.

Elsősorban a „*launchSettings.json*” fájlban el kell helyezni a „*launchUrl*”: „*scalar/v1*” sort a „*launchBrowser*”: *false*” sor alá. Valamint át kell állítani a „*false*” szót „*true*”-ra. Ezt azért kell megadni, hogy *Scalar* tesztelőprogram automatikusan elinduljon, és tudjuk tesztelni a backend szoftverünk megírt végpontjait. A teljes kódrészletet itt lehet megtalálni:

```
"https": {  
  "commandName": "Project",  
  "dotnetRunMessages": true,  
  "launchBrowser": true,  
  "launchUrl": "scalar/v1",  
  "applicationUrl": "https://localhost:7095;http://localhost:5196",  
  "environmentVariables": {  
    "ASPNETCORE_ENVIRONMENT": "Development"  
  }  
}
```

A következő lépésben a „*program.cs*” állományt kell módosítani. A módosítás sok mindent érint a nyers fájlban. Ide kell beírni a „*Scalar*” meghívást, a szolgáltatások közé fel kell venni az „*AddDbContext*”-et. Ez azt a célt szolgálja, hogy az adatbázishoz hozzá tudjunk férni. A fájl módosításai lentebb találhatóak:

```
builder.Services.AddDbContext<SzinesNegyEvszakContext>();
```

A fenti sor az adatbázis illesztése, a lentebbi kód a tesztelőprogramot hívja meg. Miután elindítottuk a programunkat a „*Scalar*” automatikusan megnyílik.

```
// Configure the HTTP request pipeline.  
if (app.Environment.IsDevelopment())  
{  
  app.MapOpenApi();  
  app.MapScalarApiReference();  
}
```

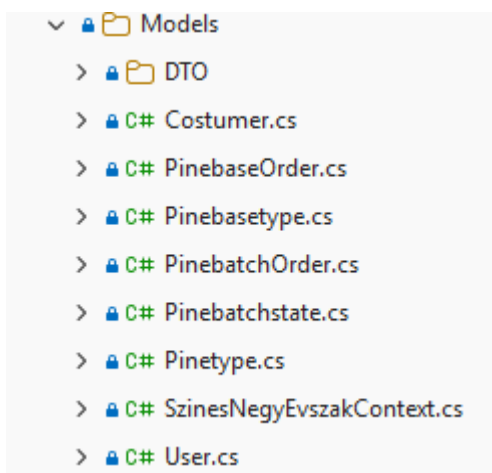
Az adatbázis migrálása:

Ahhoz, hogy az adatbázis tábláit egyben be tudjuk húzni a projektbe, és ne kelljen külön osztályokat létrehozni a táblákból, használhatjuk az előkészületekben említett „NuGet Package Manager Console”-t. Ahhoz, hogy az adatbázist megszerezzük a „Console”-ba be kell írni az integrációért felelő parancsot. Ez a parancs a következőképpen néz ki:

```
Scaffold-DbContext  
("server=localhost;database=szines_negy_evszak;user=root;password=")  
MySQL.EntityFrameworkCore -Outputdir Models -f
```

A parancsban meg kell adni az adatbázis kapcsolat létrehozásához szükséges adatokat, valamint a MySQL.EntityFrameworkCore paramétereit: a kimeneti mappát és azt, hogy az ott lévő fájlokat felülírhatja-e a migrálás közben. Viszont a parancs kiadása előtt a „WAMP” szervert futtatni kell, mert a kiadott utasítás abban a pillanatban kapcsolódik a szerverhez és lekéri az adatbázis tábláit és adatait.

Amint a szerver átadta az adatokat, és lement a migráció, a projektben megjelennek az adatbázis táblái C# osztályokba rendezve. Mindegyik tábláról készül egy „.cs” végű fájl, amely tartalmazza az adatbázisban definiált mezőneveket.



A migráció eredménye megjelenik a Solution Explorer ablakban.



Az adatbázisban létrehozott táblák, amiket az előbb kiadott parancs hatására megjelentek a C# projektben.

Az adatbázis táblák és a C# osztályok között megjelenik egy „Context.cs” fájl. Erről már volt szó korábban is. Ebben a fájlban entitásként jelennek meg az adatbázis táblái. Ezekre hivatkoznak a különböző C# osztályokba migrált változók és tulajdonságok. Erre a fájlra hivatkozik a „program.cs” fájlban a következő utasítás:

```
builder.Services.AddDbContext<SzinesNegyEvszakContext>();
```

Adatok átadása:

Ahhoz, hogy a megfelelő adatokat tudjuk küldeni a backend és a frontend között úgy nevezett DTO-kat kell definiálni a backend programban. Ezek a Data Transfer Object-ek segítenek elrejteni az adatátvitel közben a belső érzékeny adatokat, és csak a szükséges információt adják át. Egyetlen csomagban több adatot is tud küldeni, így kevesebb hálózati hívásra van szükség. A DTO-kat már nem migrálni, hanem megírni szükséges. Viszont csak azokat az adatokat érdemes bele tenni a DTO fájlba, amiket a Frontend vár-, illetve ad. A táblák, amivel a program működtethető azokból mindenképpen szükséges DTO fájlokat készíteni, hiszen ezek a csatornák a kommunikáció részei a backend felől. Ezek úgy viselkednek a backend program futtatásakor, mintha az adatbázis táblái lennének.

Ahhoz, hogy az adatbázist elérjük úgy nevezett HTTP hívásokat kell létrehozni. Az egyik tábla adatain meg is tudjuk nézni a HTTP hívások és a DTO-k közötti kapcsolatot. A DTO fájl a következőképpen néz ki:

```
public class PinebaseOrderDTO  
{  
    public int BaseId { get; set; }  
    public int? CostumerId { get; set; }  
    public int? PineTypeId { get; set; }  
    public int? Pinebasetype { get; set; }  
    public int? BaseQuantity { get; set; }  
    public bool? BaseState { get; set; }  
    public DateTime? BaseOrderedDate { get; set; }  
}
```

Ezen tulajdonságokra hivatkozik a HTTP hívás a következő módon:

```
[HttpPost]
public async Task<ActionResult> CreatePinebaseOrder([FromBody] PinebaseOrderDTO
pinebaseOrderDTO)
{
    try
    {
        if (pinebaseOrderDTO == null)
        {
            return BadRequest(new
            {
                Message = "A Koszorúalap rendelés adatai hiányoznak!"
            });
        }
        var pinebaseOrder = new PinebaseOrder
        {
            CostumerId = pinebaseOrderDTO.CostumerId,
            PineTypeId = pinebaseOrderDTO.PineTypeId,
            Pinebasetype = pinebaseOrderDTO.Pinebasetype,
            BaseQuantity = pinebaseOrderDTO.BaseQuantity,
            BaseOrderedDate = pinebaseOrderDTO.BaseOrderedDate,
            BaseState = pinebaseOrderDTO.BaseState
        };
        await _szines_negy_evszak_context.PinebaseOrders.AddAsync(pinebaseOrder);
        await _szines_negy_evszak_context.SaveChangesAsync();
        return Ok(new
        {
            Message = "A koszorúalap rendelést sikeresen rögzítettük",
            result = pinebaseOrder
        });
    }
    catch (Exception ex)
    {
        var realmessage = ex.InnerException != null
            ? ex.InnerException.Message
            : ex.Message;
        return BadRequest(realmessage);
    }
}
```

A fentebb látható forráskódban félkövérrel kiemelt rész a kapcsolat a DTO és a Controller között. A Controller egy olyan vezérlő, amiben a HTTP hívásokat kell megírni.

Ezek a hívások közvetlenül kapcsolódnak a Frontend-hez, és az adatlekérdezés-, illetve az adatfeltöltés végrehajtható. Az AddDbContext szolgáltatás segítségével az adatbázisba mentés két sorral megoldható:

```
await _szines_negy_evszak_context.PinebaseOrders.AddAsync(pinebaseOrder);  
await _szines_negy_evszak_context.SaveChangesAsync();
```

A Backend metódusai

A Frontend és a Backend kapcsolatát forráskód szinten nem, viszont a metódusokat és azok paramétereit itt lehet megnézni.

CreateUser([FromBody] UserDTO createUserDTO){}

- **Funkció:** Új felhasználó regisztrálása esetén, POST hívás;
- **Bemenet:** UserDTO típusú adatcsomag(Username, Password);
- **Kimenet:** Sikeres, vagy sikertelen regisztráció;

Login([FromBody] UserDTO loginDTO){}

- **Funkció:** A programba való belépéskor, POST hívás;
- **Bemenet:** UserDTO típusú adatcsomag(Username, Password);
- **Kimenet:** Sikeres vagy sikertelen belépés;

GetPinetypes(){}

- **Funkció:** Lekérdezi az adatbázisban tárolt fenyőtípusokat, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Visszatérés a fenyőtípusokkal, vagy adatbázishibával;

GetPineBatchStates(){}

- **Funkció:** Lekérdezi az adatbázisban tárolt fenyőbála állapotokat, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Visszatérés a fenyőbála állapotokkal, vagy adatbázishibával;

GetPineBaseTypes(){}

- **Funkció:** Lekérdezi az adatbázisból a koszorúalap típusokat, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Visszatérés a koszorúalap típusokkal, vagy adatbázis hibával;

CreateCostumer([FromBody] CostumersDTO createCostumersDTO){}

- **Funkció:** Új vásárló regisztrálása az adatbázisba, POST hívás;
- **Bemenet:** CostumerDTO típusú adatcsomag, mely tartalmazza a vásárló:
 - Nevét;
 - Adószámát;
 - Telefonszámát;
 - Irányítószámát;
 - A várost;
 - A lakcímét;
- **Kimenet:** Sikeres regisztráció, vagy adatbázishiba;

GetAllCostumers(){}

- **Funkció:** Lekérdezi az adatbázisból a regisztrált vevők adatait, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Hiba esetén adatbázishibával, sikeres lekérdezés esetén egy csomagot ad vissza, mely tartalmazza a vevő:
 - Nevét;
 - Adószámát;
 - Telefonszámát;
 - Irányítószámát;
 - A várost;
 - A lakcímét;

GetCostumerList(){}

- **Funkció:** Lekérdezi az adatbázisból a vevő nevét és ID-ját, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Visszaadja a kért adatokat, vagy adatbázishiba lép fel;

CreatePinebatchOrder([FromBody] PinebatchOrderDTO pinebatchOrderDTO)

- **Funkció:** Új koszorúalap rendelés mentése az adatbázisba, POST hívás;
- **Bemenet:** PinebatchOrderDTO típusú adatcsomag, mely tartalmazza:
 - A fenyő típusának azonosítóját;
 - A fenyőbála állapotának azonosítóját;
 - A fenyőbála mennyiségét;
- **Kimenet:** Sikeres rendelés, vagy adatbázishiba;

GetAllPinebatchOrders(){}

- **Funkció:** Lekérdezi az adatbázisból az eddig regisztrált rendeléseket, GET hívás;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Hiba esetén adatbázishiba, amennyiben a lekérdezés sikeres volt, akkor a következő adatokat kapjuk meg egy csomagban:
 - Fenyőtípus ID-ja;
 - Fenyőbála állapot ID-ja;
 - Fenyőbála állapot állapota;
 - Fenyőbála mennyisége;

CreatePinebaseOrder([FromBody] PinebaseOrderDTO pinebaseOrderDTO)

- **Funkció:** Létrehoz az adatbázisban egy új koszorúalap rendeltet, POST hívás;
- **Bemenet:** PinebaseOrderDTO típusú adatcsomag, mely a következőket tartalmazza:
 - Vásárló ID;
 - Fenyő típus ID;
 - Fenyő alap típus;
 - Fenyő alap mennyisége;
 - Mikorra kell dátum;
 - A megrendelés állapota;
- **Kimenet:** Sikeres rögzítés, vagy adatbázishiba;

GetAllPinebaseOrder()

- **Funkció:** Lekérdezi az adatbázisból a koszorúalapok éppen aktuális állapotát
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Hiba esetén adatbázishibát dob, amennyiben a lekérdezés jó Egy adatcsomagot kapunk, amiben:
 - A vevő neve;
 - A koszorúalap határideje;
 - A fenyő típusa;
 - Az alap típusa;
 - Az alap mennyisége;
 - Az alap állapota;

UpdatePinebaseOrderState(int baselId, [FromBody] bool state)

- **Funkció:** Amikor a koszorúalap elkészül és a checkbox true állapotot vesz fel, ekkor az adatbázisban a kijelölt elem frissíti az állapotát. PUT hívás;
- **Bemenet:** baselId és a state checkbox állapota;
- **Kimenet:** A koszorúalap kész állapotának módosítása, vagy hiba esetén adatbázishiba;

A Frontend készítésének lépései

Kapcsolat a Frontend és a Backend között

A Frontend kezdőoldala az *index.html* fájl. Az oldalon div-ek, label-ek, inputok vannak elhelyezve. Itt jelentkezik be a felhasználó, vagy regisztrálhat. Az *index.js* állományban találhatóak a parancsgombok eseménykezelője, valamint a táblázatok kirajzolásához szükséges metódusok. A következő oldalon különböző lehetőségek közül lehet választani. A fenyőbála rendelés, vagy a koszorúalap készre jelölése, esetleg új vásárló regisztrálása a cél, minden eseménykezelő függvény az előzőleg említett fájlban található.

A többi *.html fájlban is vannak label-ek, inputok, „Click” események, valamint a már az adatbázisba mentett adatok. A vevők, a koszorúalap rendelések, és a fenyőbála rendelések adatait - a lehetőségekre kattintva - a következő oldalon táblázatos rendszerben megjeleníti a weboldal.

Minden „Click” eseménynél az *index.js* állományban összegyűjtjük a Backend által kért adatokat. A megfelelő esemény meghívása a megfelelő adatokkal az már az *api.js* állományra mutat, ahol a Backend hívás történik. Ebben a fájlban találhatóak a különböző Backend hívások. Ezek közvetlenül kapcsolódnak a Backend programhoz. Azokat az adatokat, melyeket a Backend metódusai igényelnek, ezek a Frontend függvények biztosítják.

A Frontend metódusai

Login(username, password)

- **Funkció:** A paraméterben megadott adatokat továbbítja a Backend számára;
- **Bemenet:** username, password;
- **Kimenet:** Lehet adatbázishiba, vagy felhasználó-jelszó páros hiba.
Máskülönben belépés;

RegisterNewUser(username, password)

- **Funkció:** A paraméterben megadott adatokat továbbítja a Backend felé;
- **Bemenet:** username, password;
- **Kimenet:** Adatbázis hiba, vagy sikeres regisztráció;

GetPineTypes()

- **Funkció:** Lekéri az adatbázisból a fenyőtípusokat;
- **Bemenet:** Nincs szükség adatra;
- **Kimenet:** Adatbázishiba, vagy a fenyőtípusok adatai;

GetPineBatchStates()

- **Funkció:** Lekéri az adatbázisból a fenyőbálák állapotait;
- **Bemenet:** Nincs szükség bemeneti adatokra;
- **Kimenet:** Adatbázishiba, vagy a fenyőbálák állapotai;

GetPineBatchOrder()

- **Funkció:** Lekérdezi az adatbázisból a fenyőbála rendeléseket;
- **Bemenet:** Nincs szükség bemeneti adatokra;
- **Kimenet:** Adatbázishiba, vagy a fenyőbála rendelések adatai;

RegisterNewPineBatchOrder(pine_type, batch_state, quantity)

- **Funkció:** Új fenyőbála rendelés paramétereit átadja a Backend-nek;
- **Bemenet:** A fenyőtípus, a fenyőbála állapotjelzője és a mennyisége;
- **Kimenet:** Adatbázishiba, vagy az adatok mentése az adatbázisba;

GetPineBaseTypes()

- **Funkció:** Lekérdezi a koszorúalapok típusát;
- **Bemenet:** Nincs szükség bemeneti adatokra;
- **Kimenet:** Adatbázishiba, vagy a koszorúalapok típusainak listája;

RegisterNewCostumer(costumer_name, costumer_postal_code, costumer_city, costumer_address, costumer_phonenumber, costumer_taxnumber)

- **Funkció:** Egy új vásárló regisztrálása az adatbázisban;
- **Bemenet:** Az új vásárló fontos adatait kell megadni
 - Név;
 - Irányítószám;
 - Város;
 - Cím;
 - Telefonszám;
 - Adószám;
- **Kimenet:** Adatbázishiba, vagy az új vásárló sikeres regisztrációja;

GetAllCostumers()

- **Funkció:** Az összes vásárló adatait lekérdezni az adatbázisból;
- **Bemenet:** Nincs szükség bemeneti adatokra;
- **Kimenet:** Egy listában megkapjuk az összes vásárló adatait, vagy adatbázishiba;

GetCostumersList()

- **Funkció:** Lekérdezi az adatbázisból a vásárlók neveit;
- **Bemenet:** Nincsenek bemeneti adatok;
- **Kimenet:** Az adatbázisból kapott vevői lista, vagy adatbázishiba;

RegisterNewPinebaseOrder(costumerId, pineTypeld, pinebasetype, baseQuantity, baseOrderedDate, baseState)

- **Funkció:** Új koszorúalap megrendelés regisztrálása az adatbázisba;
- **Bemenet:** A koszorúalap megrendeléséhez szükséges adatok
 - Vevő;
 - Fenyőtípus;
 - Koszorúalap típus;
 - Koszorúalap mennyisége;
 - Elkészülési határidő;
 - Koszorúalap állapota;
- **Kimenet:** Adatbázishiba, vagy a koszorúalap rendelés adatbázisba mentése;

GetAllPinebaseOrders()

- **Funkció:** Lekérdezi az adatbázisban tárolt koszorúalapok adatait;
- **Bemenet:** Nincs szükség adatokra;
- **Kimenet:** Adatbázishiba, vagy a koszorúalapok adatainak listája
 - Vevő;
 - Fenyőtípus;
 - Koszorúalap típus;
 - Koszorúalap mennyisége;
 - Elkészülési határidő;
 - Koszorúalap állapota;

UpdatePinebaseOrderState(baseId, baseState)

- **Funkció:** Frissíti az adatbázisban a koszorúalapok állapotát;
- **Bemenet:** A kiválasztott koszorúalap azonosítója, és az állapota;
- **Kimenet:** Az adatbázisban frissíti a kiválasztott koszorúalap állapotát;

Az alkalmazás indításához szükséges szoftverek

A Frontend eszközei:

A Frontendet a Visual Studio Code névre keresztelt fejlesztői környezetben készítettük el JavaScript és HTML nyelven. Ez az eszköz segíti a fejlesztőt olyan bővítményekkel, mint a Live Server, amivel a fejlesztés folyamán, akár már egy billentyűleütésnyi változást is láthatóvá tesz. A program sokféle fájlkiterjesztéssel tud dolgozni, amennyiben olyat szeretnénk használni, ami nincs a listában, akkor egy másik bővítmény telepítésével meg is oldható. Ennél a projektnél csak a Live Serverre volt szükség, ugyanis a fejlesztőkörnyezet alapból ismeri a .html-t, és a .js-t fájlkiterjesztéseket.

A Backend eszközei:

A Backendet a Microsoft Visual Studio néven futó fejlesztőkörnyezet segítségével alkottuk meg. Ezt a programot C# nyelven hoztuk létre. Az adatbázis és a Backend szoftver közötti kapcsolat létrehozásához ide is kell telepíteni néhány kiegészítőt. Ebben nagyon sokat tud segíteni a NuGet Package Manager. Viszont ezt már korábban említettem. Az adatbázis kapcsolat létrehozását is az egyik ilyen bővítmény segíti egy úgynevezett kapcsolati karakterlánc segítségével, ami a következőképpen néz ki:

```
Scaffold-DbContext  
("server=localhost;database=szines_negy_evszak;user=root;password=")  
MySQL.EntityFrameworkCore -OutputDir Models -f
```

A WAMP szerver:

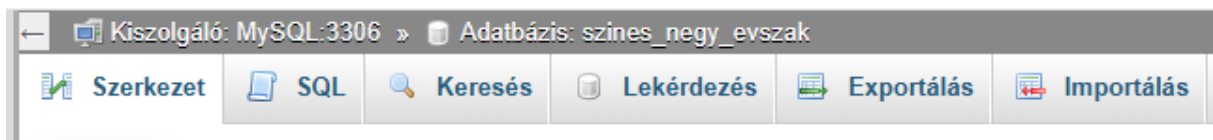
Ebben a szerverprogramban lehet létrehozni azt az adatbázist, amihez a Backend programunk csatlakozik. A WAMP letöltése teljesen ingyenes, valamint telepítés után azonnal használható. A program tartalmazza az Apache servert, MySQL-, és a MariaDB adatbáziskezelőket, valamint a PHP programozási nyelvet.

Az adatbázis:

Az előbb említett WAMP szerver segítségével lehet létrehozni az adatbázist. Az adatbázis tábláinak létrehozásához egy fájlban már megvannak a parancsok. Ahhoz, hogy ezeket importálhassuk egy kattintással, először létre kell hozni magát az adatbázist. A következő SQL utasítás pont ezt hajtja végre.

```
CREATE DATABASE szines_negy_evszak;
```

Miután lefuttattuk a fenti kódot létrejön az adatbázis a WAMP szerveren. Az adatbázis kiválasztása után az importálás fülre kattintva be kell másolni a már kész utasításokat.



A fenti képen lehet látni a kiválasztott adatbázist, és az importáláshoz szükséges parancsgombot. Az előzőekben leírt folyamatot végig járva az adatbázisban megjelennek a táblák.

A webalkalmazás elindítása:

A WAMP szerver elindítása után – mivel az adatbázist – már tartalmazza más dolgunk ebben a programban nincs. Ezután a Backend programot kell megnyitni a Visual Studio-ban, és elindítani a programot. Abban az esetben, amikor már a háttérprogramok futnak el lehet indítani a Frontend fejlesztőkörnyezetét a Visual Studio Code-ot. Itt ki kell választani az *index.html* állományt és indítani a már említett Live Server-t.

Ekkor az alapértelmezett böngészőnk elindul, és megjelenik az alkalmazásunk első oldala. Ahogyan a programot megírtuk, úgy is kell viselkednie. A fejlesztés során előkerült hibákat mostanra kijavítottuk, így már csak a jól működő program előnyeit kell élvezni.